

NATIONAL UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Applied Sciences · Department of Computer Science

Solution Book 1

SCS 1101, October 2024 (Mr J. Mutengeni)

SCS 1101 Introduction to Computer Science and Programming

Past-paper solutions with marks allocated, algorithms, textbook flowcharts, exam-ready code and a civil & water engineering lens

Answer any 4 of 5 · 25 marks each · 3 hours · no calculators

Book 1 of 8 · Prepared October 2026

Index — tap a tab or a line to jump

Every chapter starts with the coloured tab bar below; tap any tab to jump straight to that question. The page footer has a link back to this index. In Word, the Navigation Pane shows the same chapters.

Guide How to use this book

Glance At a glance

Q1 Question 1 [25]

- ▸ 1(a) Explain these terms [2 each = 10]
- ▸ 1(b) if, if-else and else-if ladder, with flowcharts and code [5 each = 15]

Q2 Question 2 [25]

- ▸ 2(a) Binary equivalent of 156_{16} [2]
- ▸ 2(b) Three types of programming errors [3]
- ▸ 2(c) $(BAE)_{16}$ + $(DAD)_{16}$ in octal [5]
- ▸ 2(d) Program: 10 integers into an array, then their average [15]

Q3 Question 3 [25]

- ▸ 3(a)(i) What is a Local Area Network? [2]
- ▸ 3(a)(ii) Two topologies for the 16-station lab, with labelled diagrams [8]
- ▸ 3(b) Program: ZESA electricity bill [15]

Q4 Question 4 [25]

- ▸ 4(a) Five functions of the operating system [5]
- ▸ 4(b) Identify the errors in the program [10]
- ▸ 4(c) Compare and contrast databases and cloud computing [10]

Q5 Question 5 [25]

- ▸ 5(a) Pass by value vs pass by reference [5]
- ▸ 5(b) Five advantages of cloud storage [10]
- ▸ 5(c) Program: recursive Fibonacci series [10]

Refs Sources for this book

Links Source register

How to use this book

This book is one of eight stand-alone solution books for NUST SCS 1101 past papers. It solves **SCS 1101, October 2024 (Mr J. Mutengeni)** question by question, with the marks the examiner allocates shown in square brackets, e.g. [2].

The six-block answer template

Every answer in this book (and in all eight books of the set) is built from the same six blocks, in the same order. Theory answers use blocks 1, 2, 5 and 6; programming answers use all six.

1. **Like you're 3** — one or two sentences with a toy, a bath or a water tap.
2. **Model answer** — the exam wording, with the mark for each point in brackets, e.g. [2]. Write it like this and you collect every mark.
3. **Algorithm + code** — numbered pseudocode first, then C code that follows the **H-D-I-P-O-R** recipe.
4. **Flowchart or structure chart** — textbook (ANSI) symbols. Programs built from functions also get a structure (hierarchy) chart.
5. **Engineering lens** — the same idea in a civil and water engineering setting, using the profession's vocabulary.
6. **Examiner's tip** — where students usually lose marks.

The H-D-I-P-O-R code recipe — "**H**appy **D**ucks **I**n **P**onds **O**nly **R**elax". Every exam program is these six steps, in this order:

```c
/* H - Header: what the program does, and the libraries */
#include <stdio.h>

int main(void)
{
    /* D - Declare: every variable, with a sensible type and a starting value */
    /* I - Input:   printf a prompt, then scanf with & */
    /* P - Process: the if-ladder, loop or formula */
    /* O - Output:  printf the result with units and decimals */
    /* R - Return:  */ return 0;
}
```

The recipe is worth marks by itself: examiners give separate marks for headers, declarations, input, processing and output. A half-finished program laid out this way still scores; a clever one-liner without structure often does not.

Algorithm style used in every book:

```
Step 1: Start
Step 2: Read <inputs>
Step 3: <process, one action per step>
Step 4: Display <outputs>
Step 5: Stop
```

Flowchart symbols (ANSI):

Symbol	Shape	Use	Colour in this book
Terminator	Rounded box (stadium)	Start / Stop	Green

Symbol	Shape	Use	Colour in this book
Input/Output	Parallelogram	Read or display	Blue
Process	Rectangle	Calculation or assignment	Amber
Decision	Diamond	Yes/no test, two exits	Red
Predefined process	Rectangle with double sides	Call to a function	Purple
Flow line	Arrow	Order of steps	Dark grey arrow

The smallest possible program, drawn with these symbols:

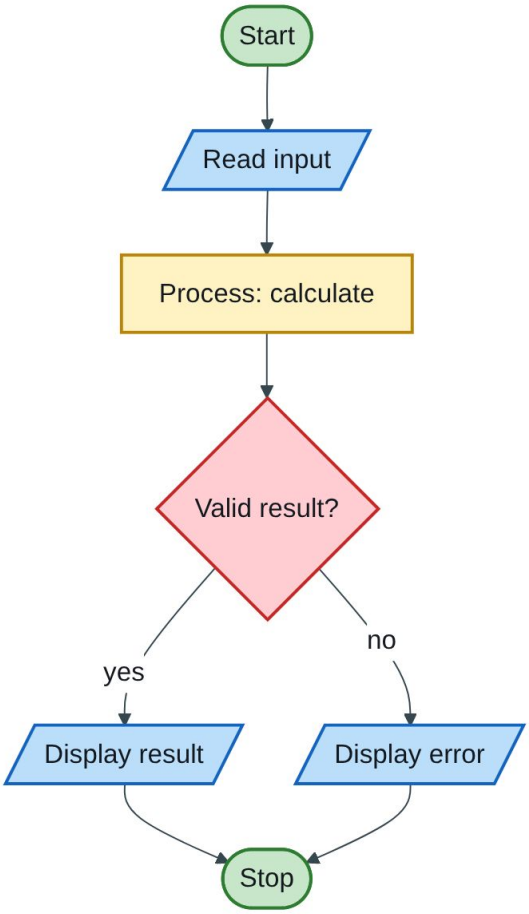


Figure 1.1 — Flowchart

The engineering lens. Programming questions get a civil and water engineering version where it fits, for example:

- tariff ladders become **increasing block tariffs** for water;
- array statistics become **rainfall** or **concrete cube strength** analysis;
- geometry programs become **cross-section, wetted perimeter and hydraulic radius** calculations.

Write the exam version in the exam; the engineering version is for understanding and for your later modules.

How the solutions were researched

Every fact a solution relies on is traced to a source in one of four tiers; each source has a short key (e.g. [MIT-2]) so answers can cite it without long links.

1. **Tier 1 — the course itself.** The SCS 1101 outline and the past papers decide *what* to answer and how marks are split.

2. **Tier 2 — university teaching material.** MIT OpenCourseWare and UCLA course pages for programming and algorithms; university histories for milestones.
3. **Tier 3 — standards and research.** NIST for cloud definitions; peer-reviewed and arXiv papers, including Wits research on cloud adoption in South Africa.
4. **Tier 4 — standard textbook knowledge.** Definitions every textbook agrees on (OS functions, topologies, printers) are written from the textbooks your outline lists, without a web citation.

A claim is only cited when a source was actually opened; anything from tier 4 stays general and uncited.

At a glance

3 hours, answer any 4 of 5 questions, 25 marks each, **no calculators**. Pick Q2, Q3 and Q5 first (long programs you can prepare), then Q1 or Q4.

Question	What it tests	Marks split	Pick?
Q1	5 definitions; if / if-else / else-if with flowcharts	10 theory + 15 code and charts	Yes, if your flowcharts are fast
Q2	Hex to binary; error types; hex addition to octal; array average	10 by hand + 15 program	Yes, highly predictable
Q3	LAN and topologies; ZESA tariff program	10 theory + 15 program	Yes
Q4	OS functions; spot 10 errors; databases vs cloud	25 theory and reading	Good backup
Q5	Value vs reference; cloud storage; recursive Fibonacci	15 theory + 10 program	Yes

Each answer below follows the library template: Like you're 3 → model answer with marks → algorithm and code → flowchart → engineering lens → examiner's tip.

Question 1 [25]

1(a) Explain these terms [2 each = 10]

Like you're 3: a DBMS is a librarian for data; an IP address is a house number for a computer; SaaS is renting a toy instead of buying it; a byte is a box of 8 light switches; a variable is a labelled jar.

Term	Model answer [1 for the definition + 1 for the detail or example]
DBMS	Database Management System: software that creates, stores, retrieves, updates and secures the data in a database, and controls shared access by many users [1]. Examples: MySQL, Microsoft Access, Oracle [1].
IP address	A unique numerical label given to every device on a network that uses the Internet Protocol, so data packets can be delivered to it [1]. IPv4 uses 32 bits, e.g. 192.168.1.10; IPv6 uses 128 bits [1].
SaaS	Software as a Service: a cloud model in which the provider runs the application on its own infrastructure and users reach it over the internet, usually through a browser, without managing servers or the operating system [1]. Examples: Gmail, Microsoft 365 [1]. (STD-1)
Byte	A group of 8 bits; the smallest unit of memory that has its own address [1]. One byte stores one character, e.g. 'A' = 01000001; 1 KB = 1024 bytes [1].
Variable	A named memory location whose value can change while the program runs; it has a data type that fixes its size and what it may hold [1]. Example: `int count = 0;` [1].

Engineering lens: a water utility keeps its asset register (pipes, valves, hydrants, meters) in a **DBMS** linked to GIS; every SCADA outstation at a pump station or reservoir has an **IP address** so the control room can poll its telemetry.

1(b) if, if-else and else-if ladder, with flowcharts and code [5 each = 15]

Like you're 3: `if` is one gate: open it only if the answer is yes. `if-else` is a fork in the road: yes goes left, no goes right. The ladder is a row of gates: you go through the *first* one that says yes.

Marks for each part: syntax or explanation [1], flowchart [2], working code snippet [2].

(i) `if` statement — runs a block only when the condition is true; otherwise the block is skipped.

```c
if (condition) {
    statements;          /* run only when condition is true */
}

/* Example: chlorine residual check at a treatment works */
if (residual < 0.2) {
    printf("Low chlorine residual: increase dosing\n");
}
```

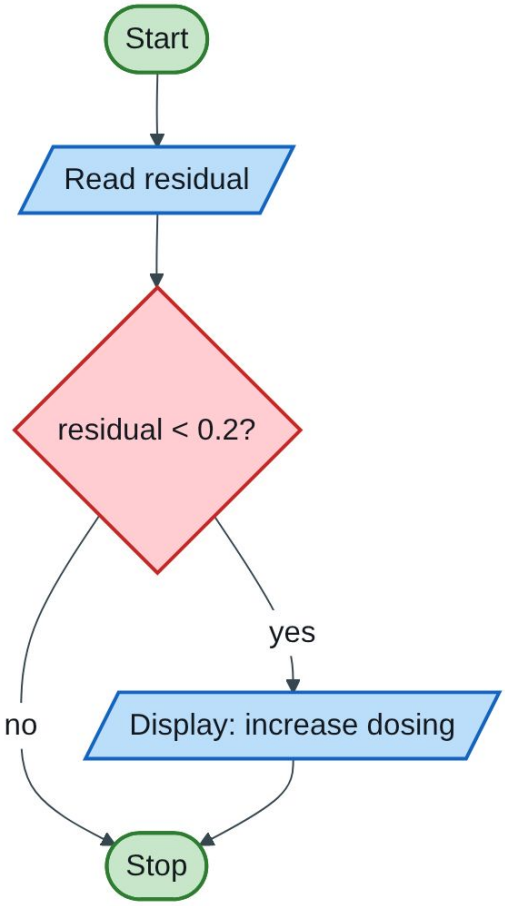


Figure 1.2 — Flowchart

(ii) `if-else` statement — chooses between exactly two blocks: one runs when true, the other when false.

```c
if (condition) {
    statements_1;        /* condition true */
} else {
    statements_2;        /* condition false */
}

/* Example: pump control from tank level (m) */
if (level >= 4.5) {
    printf("Tank full: STOP pump\n");
} else {
    printf("Tank filling: RUN pump\n");
}
```

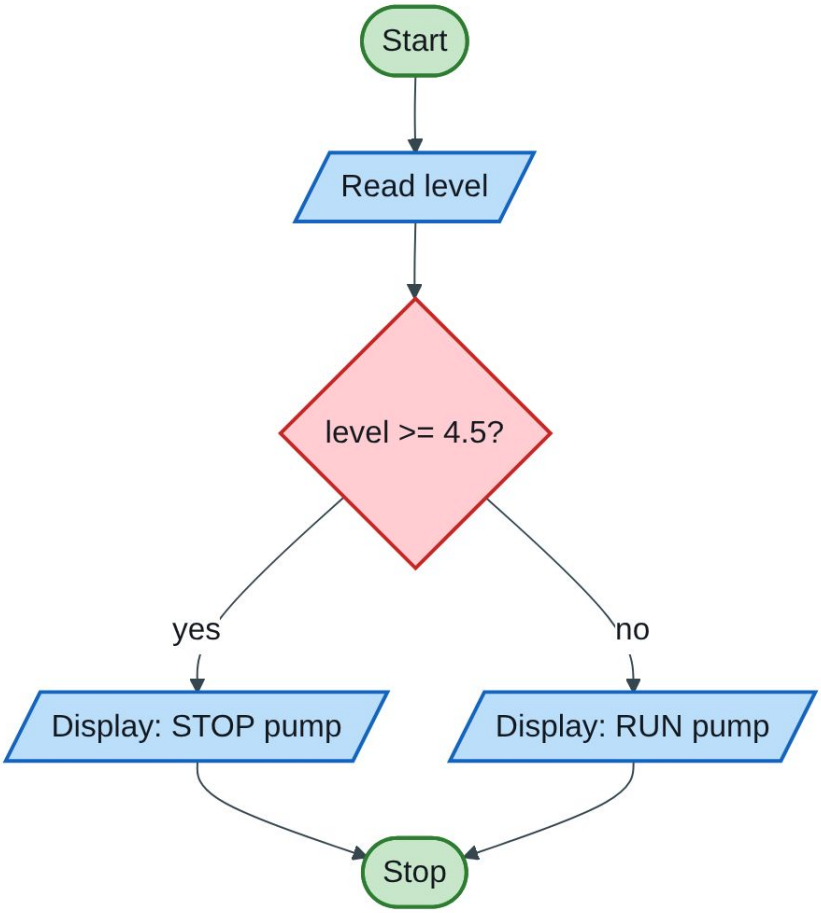


Figure 1.3 — Flowchart

(iii) `if … else if … else` ladder — tests conditions one after another; the first true one runs and the rest are skipped; the final `else` catches everything left.

```
if (cond1) {
    block1;
} else if (cond2) {
    block2;
} else if (cond3) {
    block3;
} else {
    block4;              /* none of the above */
}

/* Example: reservoir alarm from % full */
if (pct >= 90)
    printf("Overflow warning\n");
else if (pct >= 50)
    printf("Normal\n");
else if (pct >= 20)
    printf("Low: restrict supply\n");
else
    printf("Critical: emergency supply\n");
```

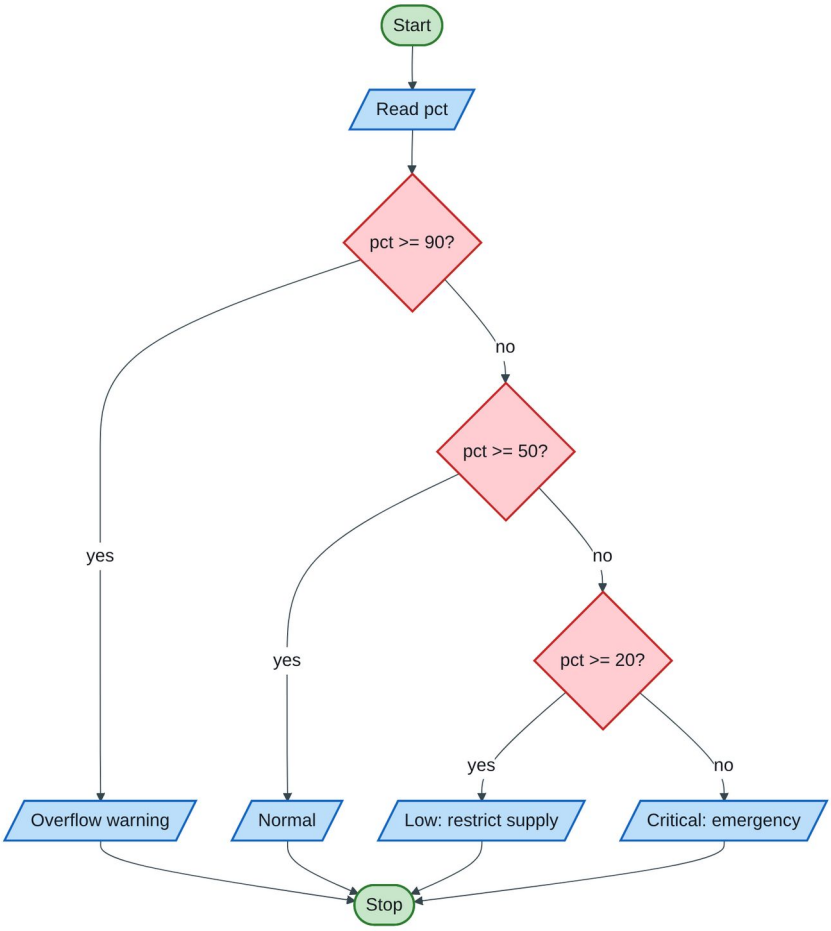


Figure 1.4 — Flowchart

Examiner's tip: draw the diamond with **two labelled exits (yes/no)** every time; an unlabelled decision loses the flowchart mark. In the ladder, test from the highest value down, so each `else if` can rely on the earlier tests having failed.

Question 2 [25]

2(a) Binary equivalent of 156_{16} [2]

Like you're 3: every hex digit is a little box that opens into exactly 4 binary switches.

Hex digit	1	5	6
4-bit binary	0001	0101	0110

156_{16} = 0001 0101 0110$_2$ = **101010110_2** [1 method, 1 answer]. Check: 1×256 + 5×16 + 6 = 342, and 101010110_2 = 256 + 64 + 16 + 4 + 2 = 342.

2(b) Three types of programming errors [3]

1. **Syntax error** — breaks the grammar of the language, so the program will not compile, e.g. a missing `;` [1].
2. **Run-time error** — the program compiles but fails while running, e.g. division by zero or reading through a `NULL` pointer [1].
3. **Logic error** — the program runs but gives the wrong answer, e.g. using `<` where `<=` was needed [1].

(Some lecturers also accept **semantic** and **linker** errors; Book 2, Q3 covers four types.)

2(c) $(BAE)_{16}$ + $(DAD)_{16}$ in octal [5]

Like you're 3: add like normal sums, but you only carry when you reach 16, not 10. Then repack the answer into groups of 3 switches for octal.

Step 1 — add in hex, right to left [2]. Digit values: A = 10, B = 11, D = 13, E = 14.

Column	Working	Write	Carry
16^0	E + D = 14 + 13 = 27 = 16 + 11	B	1
16^1	A + A + 1 = 10 + 10 + 1 = 21 = 16 + 5	5	1
16^2	B + D + 1 = 11 + 13 + 1 = 25 = 16 + 9	9	1
16^3	carry	1	—

BAE + DAD = **$195B_{16}$**

Step 2 — hex to binary, 4 bits per digit [1]: 1 = 0001, 9 = 1001, 5 = 0101, B = 1011 → 0001 1001 0101 1011.

Step 3 — regroup in 3s from the right, then read octal [2]:

Groups of 3	001	100	101	011	011
Octal digit	1	4	5	3	3

Answer: 14533_8. Check in decimal: BAE = 2990, DAD = 3501, sum = 6491; and 1×4096 + 4×512 + 5×64 + 3×8 + 3 = 6491.

2(d) Program: 10 integers into an array, then their average [15]

Like you're 3: ten egg cups in a row. Put one number in each cup, add all the cups, then share the total equally by 10.

Algorithm:

```
Step 1: Start
Step 2: Set sum = 0
Step 3: For i = 0 to 9: read values[i]
Step 4: For i = 0 to 9: sum = sum + values[i]
Step 5: average = sum / 10   (as a decimal)
Step 6: Display sum and average
Step 7: Stop
```

Flowchart:

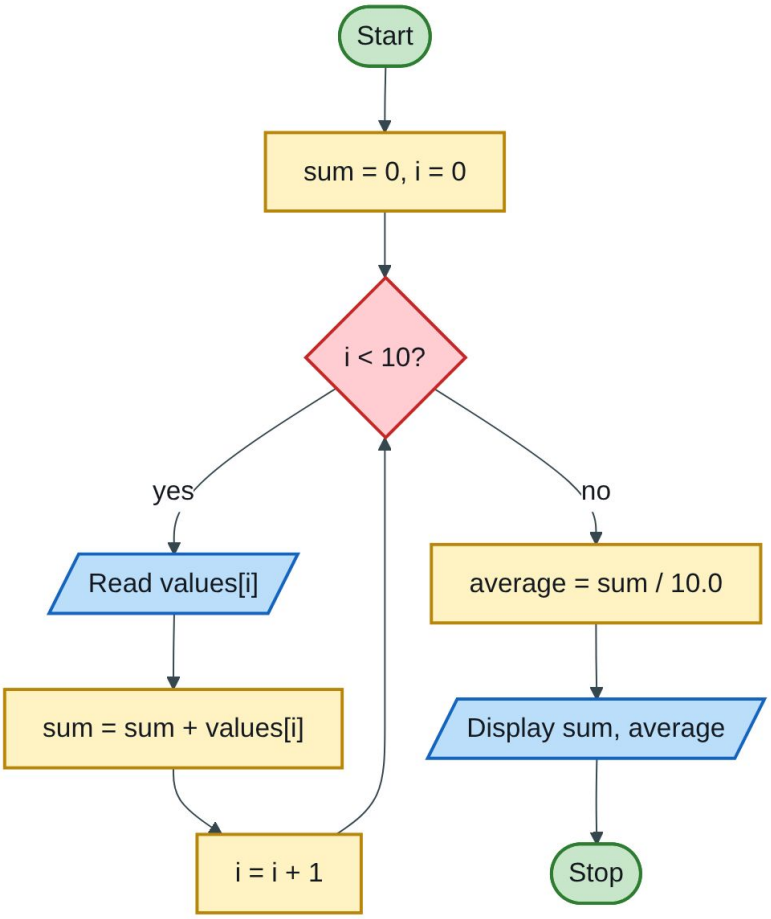


Figure 1.5 — Flowchart

(The flowchart reads and adds in one loop; the code below uses two loops to show the array clearly. Both are correct.)

Code (H-D-I-P-O-R):

```c
/* H: read 10 integers into an array, print their sum and average */
#include <stdio.h>
#define N 10

int main(void)
{
    int values[N];                  /* D: the array of 10 boxes */
```

```c
    int i, sum = 0;
    double average;

    for (i = 0; i < N; i++) {          /* I: fill the array */
        printf("Enter value %d: ", i + 1);
        scanf("%d", &values[i]);
    }

    for (i = 0; i < N; i++)            /* P: add them up */
        sum += values[i];
    average = (double)sum / N;         /* cast, or 55/10 would give 5 */

    printf("Sum     = %d\n", sum);     /* O */
    printf("Average = %.2f\n", average);
    return 0;                          /* R */
}
```

Test: entering 1 to 10 prints `Sum = 55` and `Average = 5.50`.

Marking guide [15]: header and `main` [2]; array and variables declared and `sum` initialised to 0 [3]; input loop with prompt and `&values[i]` [4]; summing loop [3]; average as a decimal with a cast [2]; clear output [1].

Engineering lens: replace the 10 integers with 10 daily **water meter readings** (m^3/day) from a district metered area. The average is the **average daily demand (ADD)**, the starting figure for sizing reservoirs and checking leakage against the night flow.

Examiner's tip: `sum / N` with two `int`s throws away the decimals. Always write `(double)sum / N` or `sum / 10.0`.

Question 3 [25]

3(a)(i) What is a Local Area Network? [2]

Like you're 3: all the computers in one classroom holding hands so they can share toys.

A LAN is a computer network that connects devices within a small geographical area, such as one lab, building or campus [1], so they can share resources (files, printers, internet connection) at high speed, usually owned and managed by one organisation [1].

3(a)(ii) Two topologies for the 16-station lab, with labelled diagrams [8]

Marks for each topology: labelled diagram [2], description [1], one advantage or disadvantage [1].

Topology 1 — Star. Every station has its own cable to a central switch (or hub); all traffic passes through the centre.

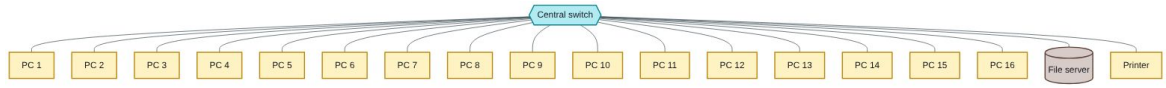


Figure 1.6 — Flowchart

- Advantage: one broken cable affects only one PC, and faults are easy to find.
- Disadvantage: if the central switch fails, the whole lab is down; it uses more cable.

Topology 2 — Bus. All 16 stations tap onto one shared backbone cable, with a terminator at each end to stop signals bouncing back.

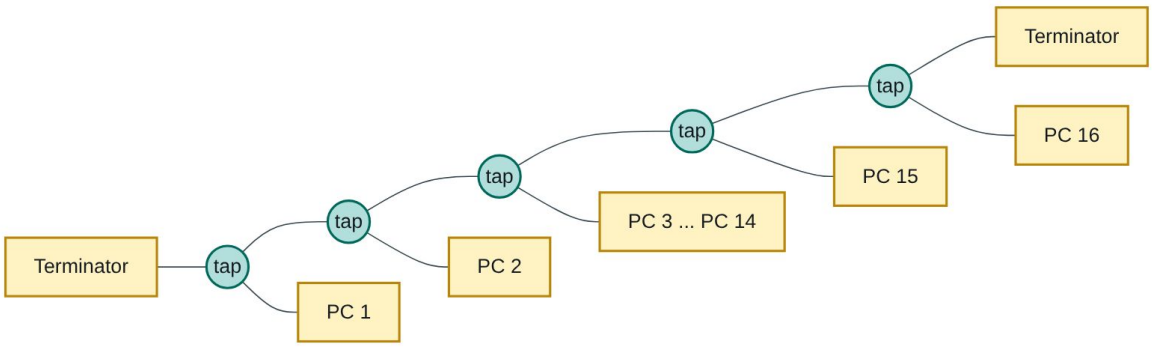


Figure 1.7 — Flowchart

Label the long horizontal line **backbone cable** in the exam.

- Advantage: cheap and simple, with the least cable.
- Disadvantage: a break in the backbone stops the whole network, and it slows down as more stations transmit (collisions).

(A ring, where each PC connects to the next in a closed loop, is also acceptable as the second topology. Star is the best choice for a modern lab.)

3(b) Program: ZESA electricity bill [15]

Like you're 3: the first 200 sweets cost 80c each, the next 100 cost 90c, and every sweet after that costs $1. Then you pay $10 just for having the sweet jar, and if the bill is huge (over $400) you pay 15% extra.

State your reading of the question (examiners accept a sensible, stated assumption): the $10 meter charge is added to every bill, and the 15% surcharge applies when that total exceeds $400.

Algorithm:

```
Step 1: Start
Step 2: Read units
Step 3: If units <= 200:      energy = units x 0.80
        Else if units <= 300: energy = 200 x 0.80 + (units - 200) x 0.90
        Else:                 energy = 200 x 0.80 + 100 x 0.90 + (units - 300) x 1.00
Step 4: total = energy + 10
Step 5: If total > 400: total = total + 0.15 x total
Step 6: Display energy charge, meter charge, total
Step 7: Stop
```

Flowchart:

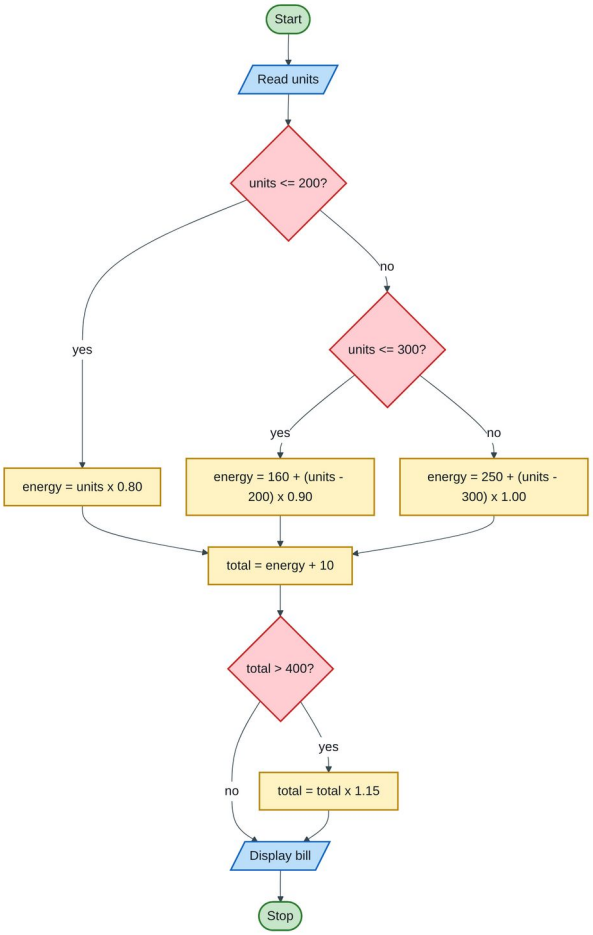


Figure 1.8 — Flowchart

(160 = 200 × 0.80 and 250 = 160 + 100 × 0.90: the full cost of the lower blocks.)

Code (H-D-I-P-O-R):

```c
/* H: ZESA bill = block tariff + $10 meter charge + 15% surcharge over $400 */
#include <stdio.h>

int main(void)
{
    double units, energy, total, surcharge = 0.0;        /* D */

    printf("Enter units consumed: ");                    /* I */
```

```c
    scanf("%lf", &units);
    if (units < 0) {
        printf("Invalid number of units\n");
        return 1;
    }

    if (units <= 200)                                   /* P: the blocks */
        energy = units * 0.80;
    else if (units <= 300)
        energy = 200 * 0.80 + (units - 200) * 0.90;
    else
        energy = 200 * 0.80 + 100 * 0.90 + (units - 300) * 1.00;

    total = energy + 10.00;                             /* meter charge */
    if (total > 400) {
        surcharge = total * 0.15;
        total += surcharge;
    }

    printf("Energy charge : $%8.2f\n", energy);         /* O */
    printf("Meter charge  : $%8.2f\n", 10.00);
    printf("Surcharge     : $%8.2f\n", surcharge);
    printf("Amount due    : $%8.2f\n", total);
    return 0;                                           /* R */
}
```

Test table — always include one value in each block:

Units	Energy charge	+ $10	Surcharge?	Amount due
150	150 × 0.80 = 120.00	130.00	No	**$130.00**
250	160 + 50 × 0.90 = 205.00	215.00	No	**$215.00**
350	250 + 50 × 1.00 = 300.00	310.00	No	**$310.00**
600	250 + 300 × 1.00 = 550.00	560.00	Yes, 84.00	**$644.00**

If your lecturer reads “minimum of $10” as “no bill is below $10”, change one line: replace `total = energy + 10.00;` with `total = (energy < 10) ? 10 : energy;`.

Marking guide [15]: header, `main`, declarations [2]; prompt and `scanf` with `&` [2]; correct three-block ladder [5]; meter charge [2]; surcharge test [2]; formatted output [2].

Engineering lens: this is exactly an **increasing block tariff (IBT)**, the structure many municipalities use for water. Change kWh to m^3 and the rates to $/$m^3$; the first block is the **lifeline block** (cheap water for basic needs), the meter charge is the **fixed (standing) charge**, and the dearer upper blocks are a **water demand management** tool that discourages waste.

Examiner’s tip: the most common error is charging *all* units at the top rate (e.g. 350 × $1). Each block is charged only for the units that fall inside it, so lower blocks are always added in full.

Question 4 [25]

4(a) Five functions of the operating system [5]

Like you're 3: the operating system is the head teacher. It decides who uses the classroom, keeps the cupboards tidy and makes sure nobody breaks the rules.

Any five, 1 mark each:

1. **Process management** — creates, schedules and ends programs, sharing the CPU between them.
2. **Memory management** — gives RAM to programs and takes it back when they finish.
3. **File management** — organises files and folders on storage; controls reading, writing and deleting.
4. **Device (I/O) management** — controls printers, disks and keyboards through device drivers.
5. **Security and access control** — user accounts, passwords and permissions.
6. **User interface** — provides a GUI or command line.
7. **Error detection and accounting** — reports faults and logs resource use.

4(b) Identify the errors in the program [10]

Like you're 3: a spelling test for the computer. C is very fussy: one wrong capital letter or missing dot and it refuses to work.

Line	Error	Correction
1	`#Include` — directives are lowercase	`#include <stdio.h>`
2	Comment opened with `/*` but never closed, so everything after it is swallowed	End it with `*/`
3	`int main[ ] (` — square brackets instead of round; `(` instead of `{`	`int main(void) {`
4	`float x` — missing semicolon	`float x;`
5	`X=5.4;` — C is case-sensitive; `X` was never declared	`x = 5.4;`
6	Closing `"` missing in the string	`printf(" Welcome to C Programming ");`
7	`%d` used to print a `float`	`printf("%f\n", x);`
8	`print f` has a space; curly quotes “ ” are not valid C	`printf("%f", x += 2.5);`
9	`return 0.` — full stop instead of semicolon	`return 0; }`

Corrected program:

```c
#include <stdio.h>
/* run this program using the console pauser or add your own getch,
```

```c
   system("pause") or input loop */
int main(void) {
    float x;
    x = 5.4;
    printf(" Welcome to C Programming ");
    printf("%f\n", x);
    printf("%f", x += 2.5);
    return 0;
}
```

Marks: about 1 per error found and corrected. List at least ten points to be safe: the line 3 fault counts as two (brackets and brace).

4(c) Compare and contrast databases and cloud computing [10]

Like you're 3: a database is a very tidy toy cupboard. Cloud computing is renting a whole playroom (shelves, power, even toys) from someone else, and visiting it through the window of the internet.

Definitions [2]:

- A **database** is an organised, structured collection of related data, managed by a DBMS so it can be stored, searched and updated efficiently.
- **Cloud computing** is a model that gives on-demand network access to a shared pool of computing resources (networks, servers, storage, applications) that can be supplied and released quickly with little management effort (STD-1).

Similarities (compare) [3]:

1. Both store and manage data for many users.
2. Both need security, backup and access control.
3. They overlap: a database can itself run in the cloud as Database-as-a-Service.

Differences (contrast) [5]:

Point	Database	Cloud computing
What it is	A store of structured data plus its DBMS	A way of delivering computing resources (servers, storage, software) over the internet
Scope	Data only	Infrastructure, platforms and applications (IaaS, PaaS, SaaS)
Location	Can be on one local PC or server, no internet needed	Remote data centres, reached over a network
Cost model	Bought or licensed and run in-house (capital cost)	Pay-as-you-go subscription (operating cost)
Scaling	Limited by your own hardware	Rapid elasticity: scale up or down on demand
Example	A MySQL student-records database	Google Drive, AWS, Microsoft 365

Conclusion: they are not rivals; the cloud is *where and how* computing is delivered, and a database is *one of the things* that can be delivered that way. A survey of South African small businesses by a Wits researcher found that the relative advantage of the cloud (lower IT cost, scalability, competitiveness) is the strongest reason firms adopt it, while weak government and regulatory support is seen as a barrier (WITS-1).

Engineering lens: a water utility's **billing database** may sit on a server in its own building (a database without the cloud). Moving the same database to AWS or Azure gives remote access for field engineers and automatic backups, at the cost of a monthly bill and reliance on the internet link.

Examiner's tip: "compare and contrast" needs **both** similarities and differences. A table of differences alone caps you at about 6/10.

Question 5 [25]

5(a) Pass by value vs pass by reference [5]

Like you're 3: pass by value is giving your friend a **photocopy** of your drawing: they can scribble on it, and your drawing stays clean. Pass by reference is giving them **your actual drawing** (its address): their scribbles land on your page.

- **Pass by value** — the function receives a copy of the argument; changes inside the function do not affect the caller's variable [1].
- **Pass by reference** — the function receives the address of the variable (in C, through a pointer), so it works on the original and changes are kept [1].

```c
void addTenV(int x)  { x = x + 10; }     /* works on a copy  */
void addTenR(int *x) { *x = *x + 10; }   /* works on the original */

int main(void)
{
    int a = 5;
    addTenV(a);      /* a is still 5  */
    addTenR(&a);     /* a is now 15   */
    printf("%d\n", a);
    return 0;
}
```

Code showing both [2]. Use pass by value to protect data; use pass by reference when a function must change the caller's variable or return more than one result (e.g. `swap`, `scanf`) [1].

5(b) Five advantages of cloud storage [10]

Like you're 3: instead of keeping your toys in your own small box, you keep them in a giant, guarded toy warehouse that you can visit from any house.

Name the advantage [1] and explain it [1], five times:

1. **Access from anywhere, on any device** — files are reached over the internet from a phone, laptop or site tablet.
2. **Lower cost** — no servers or disks to buy; you pay only for the space you use (pay-as-you-go).
3. **Scalability** — storage grows or shrinks on demand, without installing new hardware.
4. **Automatic backup and disaster recovery** — copies are kept in several data centres, so a fire, theft or dead laptop does not lose the data.
5. **Collaboration and sharing** — several people edit the same file, with version history and share links.
6. **No maintenance** — the provider handles hardware, updates and much of the security.

The Wits-led study of South African small businesses lists these same benefits (easier IT management, lower IT cost, a centralised place to work together, scalability and anytime availability) and found the cloud's relative advantage to be the strongest driver of adoption (WITS-1).

Engineering lens: a design office keeps its **AutoCAD drawings and BIM models** in cloud storage, so the site engineer opens the latest revision on a tablet on site, and surveyors upload **levelling and GPS field data** the same day instead of carrying flash disks.

5(c) Program: recursive Fibonacci series [10]

Like you're 3: each new number is the two numbers before it added together: 0, 1, 1, 2, 3, 5, 8… The function asks two smaller copies of itself for help until it reaches the first two numbers, which it already knows.

The rule: fib(0) = 0, fib(1) = 1 (base cases); fib(n) = fib(n − 1) + fib(n − 2) (recursive case).

Algorithm:

```
Function fib(n):
  Step 1: If n = 0, return 0
  Step 2: If n = 1, return 1
  Step 3: Return fib(n-1) + fib(n-2)

Main:
  Step 1: Start
  Step 2: Read n (number of terms)
  Step 3: For i = 0 to n-1: display fib(i)
  Step 4: Stop
```

Flowchart of the recursive function:

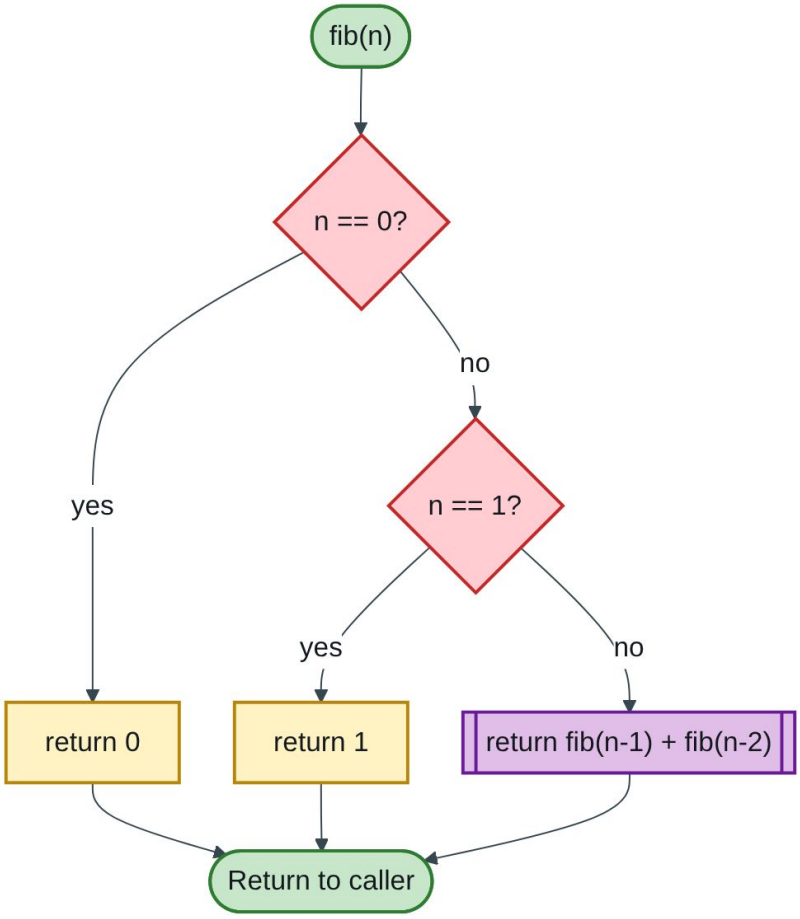


Figure 1.9 — Flowchart

Code (H-D-I-P-O-R):

```c
/* H: print the first n Fibonacci numbers using a recursive function */
#include <stdio.h>

int fib(int n);                         /* prototype */
```

```c
int main(void)
{
    int n, i;                           /* D */

    printf("How many terms? ");         /* I */
    scanf("%d", &n);

    printf("Fibonacci series: ");       /* P + O */
    for (i = 0; i < n; i++)
        printf("%d ", fib(i));
    printf("\n");
    return 0;                           /* R */
}

int fib(int n)
{
    if (n == 0) return 0;               /* base case 1 */
    if (n == 1) return 1;               /* base case 2 */
    return fib(n - 1) + fib(n - 2);     /* recursive case */
}
```

Test: n = 10 prints `0 1 1 2 3 5 8 13 21 34`.

How the calls branch for fib(4) (the call tree):

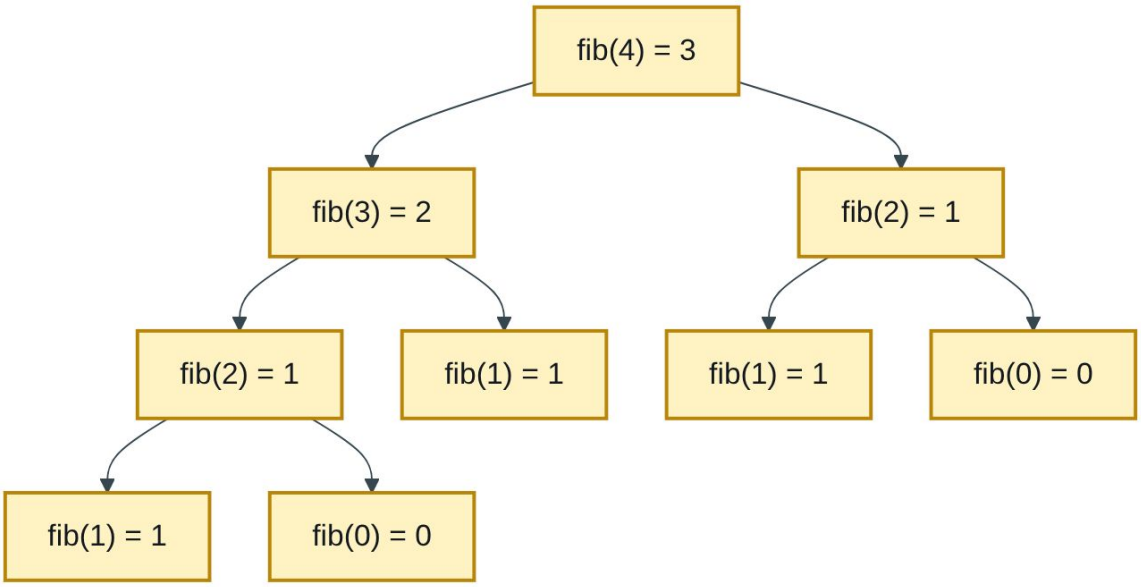


Figure 1.10 — Flowchart

Notice fib(2) is worked out twice. MIT's algorithms course uses exactly this example: the naive recursive version repeats work and takes exponential time, while storing each answer the first time (memoization) cuts it to about n steps (MIT-2). Mentioning this in one line can earn a bonus mark for understanding.

Marking guide [10]: header and prototype [1]; both base cases [2]; recursive call [2]; input of n [2]; loop printing the series [2]; tidy output [1].

Examiner's tip: a recursive function with no base case never stops and crashes with a stack overflow. Write the base cases **first**, before the recursive line.

Sources for this book

Full details of each source, with its link, are in the source register that follows.

- [STD-1] NIST SP 800-145 — SaaS definition (1a), cloud definition (4c).
- [WITS-1] Sithole (Wits) & Ruhode, arXiv:2108.10079 — cloud adoption and benefits in South Africa (4c, 5b).
- [MIT-2] MIT 6.006 Lecture 19 — naive vs memoized Fibonacci (5c).
- [NUST-1] SCS 1101 course outline — C as the course language; topics tested.
- Textbook knowledge (tier 4): number systems, OS functions, topologies, error types.

Source register

The full register for the solution-book set. Sources marked ✓ are cited in this book; the rest (MIT, UCLA and Wits material used across the set) are further reading on the same topics.

Key	Source	Institution	Used for	In this book
NUST-1	SCS 1101 course outline (uploaded)	NUST	Syllabus, assessment, C as the course language	✓
MIT-1	6.087 Practical Programming in C	MIT	C syntax, pointers, structs, files, coding style	Further reading
MIT-2	6.006 Lecture 19: Dynamic Programming I (Fibonacci)	MIT	Why naive recursive Fibonacci is slow; memoization	✓
MIT-3	Whirlwind Computer, 1944–59 (IEEE Milestone)	MIT	Magnetic-core memory, real-time computing	Further reading
MIT-4	Wang et al., *Undefined Behavior: What Happened to My Code?*	MIT CSAIL	Run-time and undefined-behaviour errors	Further reading
UCLA-1	First internet message, 29 Oct 1969	UCLA Samueli	History of networks and the internet	Further reading
UCLA-2	Birthplace of the Internet (IEEE Milestone)	UCLA	ARPANET, packet switching, first node	Further reading
UCLA-3	CS 31 Introduction to Computer Science I, syllabus	UCLA	Intro-programming scope; portability across compilers	Further reading
WITS-1	Sithole (Wits) & Ruhode, *Cloud Computing Adoption: Opportunities and Challenges for SMMEs in South Africa*, arXiv:2108.10079	Wits	Cloud benefits and barriers in Southern Africa	✓

Key	Source	Institution	Used for	In this book
WITS-2	*Leadership and the adoption of cloud computing for enterprises in Johannesburg* (2022)	Wits	Drivers and barriers of cloud adoption	Further reading
STD-1	NIST SP 800-145, *The NIST Definition of Cloud Computing*	NIST	SaaS, PaaS, IaaS; 5 characteristics; 4 deployment models	✓
RES-1	Bakasa & Pekane, *Decision Criteria for Cloud Adoption by Large Enterprises in South Africa*, arXiv:2108.09757	UCT	Security and data sovereignty as selection criteria	Further reading
RES-2	Fu et al., *Error Log Analysis in C Programming Language Courses* (ICCE 2015)	Kyushu University	The errors novice C students actually make	Further reading